Trace2Skill：将轨迹局部经验蒸馏为可迁移的智能体技能

Jingwei Ni[§,*2,3], Yihao Liu[§,*4], Xinpeng Liu[§,*4], Yutao Sun[§,*5], Mengyu Zhou[†1], Pengyu Cheng[1], Dexin Wang[1], Erchao Zhao[1], Xiaoxi Jiang[1], and Guanjun Jiang[1]

[1]阿里巴巴 Qwen 大模型应用团队
[2]苏黎世联邦理工学院（ETH Zürich）
[3]苏黎世大学
[4]北京大学
[5]浙江大学

[§]核心贡献者。通讯作者：zhoumengyu.zmy@alibaba-inc.com

Abstract

大型语言模型（LLM）智能体日益依赖领域专用技能，但人工编写这类技能难以规模化，而仅凭参数化知识生成的技能往往会遗漏关键的操作陷阱。我们提出 Trace2Skill，这是一个通过对智能体经验进行归纳推理，将大量执行轨迹并行整合到统一技能目录中的框架。Trace2Skill 既能深化现有的人工编写技能，也能从由 LLM 生成的粗略草稿中创建实用技能。实验表明，Trace2Skill 在办公工作流、数学推理和视觉问答等多个领域均有效。重要的是，演化后的技能并非只是对创建它们所用轨迹的记忆产物：它们通常能够跨模型规模、跨模型家族迁移，并泛化到分布外场景。例如，从 Qwen3.5-35B 轨迹演化而来的技能，在 WikiTableQuestions 上可使 Qwen3.5-122B 智能体的表现最高提升 57.65 个百分点。进一步分析表明，Trace2Skill 优于顺序式技能编辑和 ReasoningBank 风格的检索记忆，能将反复出现的失败和变通方案压缩为标准操作规程（SoPs），并产出无需参数更新或测试时检索即可复用的可移植技能。[1]

1 引言

基于 LLM 的智能体日益依赖技能：即编码任务流程、领域知识和操作指南的可复用文档 [Anthropic, 2026d, Zhou et al., 2026b]。随着智能体进入专门的文件与工具使用工作流，瓶颈不再只是模型能力，还包括为每个领域创建和维护高质量技能的能力 [Han et al., 2026, Li et al., 2026a, Anthropic, 2026b, Liang et al., 2026, Zhou et al., 2026b]。人工编写的技能可能有所帮助，但对不同智能体的益处并不一致：在表 1 中，官方 `xlsx` 技能提升了 122B 电子表格智能体的表现，却在同一基准上损害了 35B 智能体的表现。仅凭参数化知识生成技能同样不够稳健，因为这类草稿往往缺少实际执行轨迹所揭示的具体失败模式、变通方案和操作细节 [Li et al., 2026b, Jiang et al., 2026, Zhou et al., 2026b]。

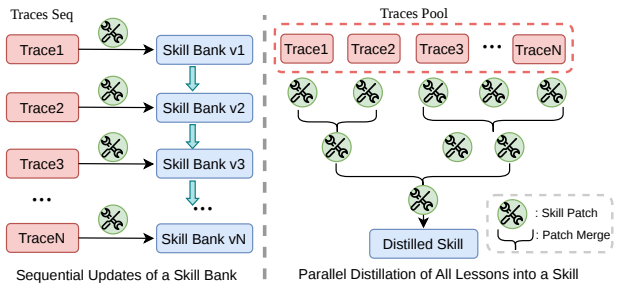


图 1: **左**：顺序式在线技能演化会在每条新轨迹到来后编辑技能。**右**：Trace2Skill 并行分析多条轨迹，并以分层方式将反复出现的经验整合为一项可移植技能。

因此，近期系统开始利用智能体经验在线演化技能或记忆 [Yang et al., 2026, Xia et al., 2026a, Alzubi et al., 2026, Zhou et al., 2026a, Jiang et al., 2026, Zhou et al., 2026b]。这一方向很有前景，但许多现有方法要么存储轨迹局部经验以供检索，要么随着新轨迹到来顺序式地编辑技能（图 1，左）。这类设计可能会将可复用知识分散在庞大的记忆或技能集合中，而顺序式编辑还会使技能的后续版本依赖此前更新的顺序 [Li, 2026]。**人类专家通常采用不同的方式**：他们审阅大量轨迹，抽象反复出现的模式，并编写可跨案例复用的精炼流程。

我们提出 Trace2Skill，一个将执行轨迹转化为可移植技能的框架。Trace2Skill 不在测试时检索单回合记忆，也不通过依赖次序的顺序式编辑来吸收轨迹，而是联合分析多条轨迹，并将反复出现的经验整合到单一技能目录中（图 1，右）。同一机制支持两种常见用例：深化现有的人工编写技能，以及从由 LLM 生成的粗略草稿中创建实用技能。这种多对一整合是对智能体经验进行的一步归纳推理 [Xiong et al., 2025, Li et al., 2025, Lin et al., 2025]，同时保留普通技能文件的可移植性。

[1]代码：https://github.com/Qwen-Applications/Trace2Skill

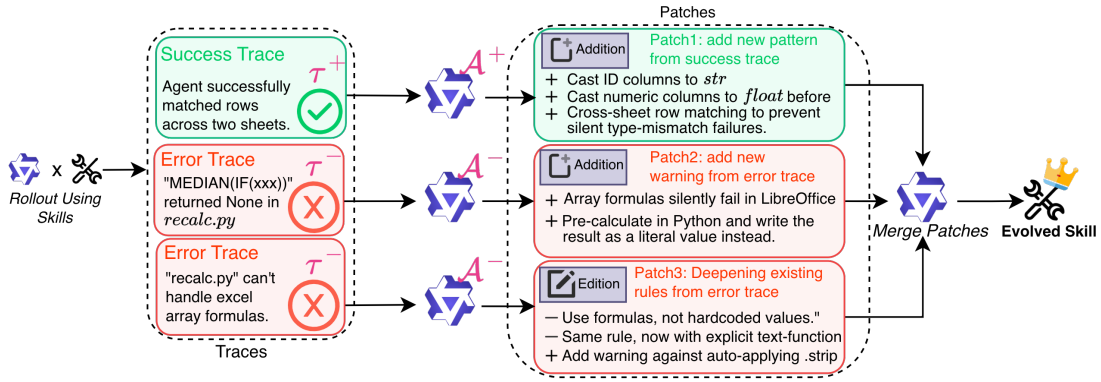


图 2: Trace2Skill 三阶段流程概览：(1) 运行冻结的智能体，收集带有成功或失败标签的轨迹；(2) 使用独立的错误分析器和成功分析器，并行提出轨迹级技能补丁；(3) 以分层方式将所有补丁合并到一个可移植的技能目录中。

实验表明，以轨迹为依据的技能能够提升性能，同时保持跨模型、基准和任务领域的可移植性。在电子表格工作流中，Trace2Skill 既能增强 Anthropic 官方的 `xlsx` 技能，也能从零创建实用技能。所得技能可以跨模型规模和家族迁移：例如，使用 Gemma-4-31B-it [Google, 2026] 演化的技能能够提升 Qwen3.5-122B [Team, 2026] 的表现。它们还能泛化到分布外（OOD）数据和任务，例如从电子表格编辑迁移到表格问答。其优势不局限于电子表格：Trace2Skill 还能提升数学推理和 DocVQA 的表现，并进一步增强 Anthropic 官方的 PDF、DOCX 和 PPTX 工作流技能。进一步分析表明：并行整合比依赖次序的技能编辑快得多，而且通常效果更强；单项蒸馏技能优于 ReasoningBank 风格的情节式检索 [Ouyang et al., 2026]；智能体分析器通过检查产物并验证修复，能够产出更好的补丁。从定性角度看，学到的补丁并非特定于轨迹的技巧，而是汇聚成可复用的 SoPs；补丁选择研究还表明，补丁的价值往往具有组合性，因此整体整合比贪心式局部选择更可靠。

我们的贡献包括：(1) **Trace2Skill**，一个用于自动创建和深化技能的框架。它模拟人类编写技能的方式：先通过广泛的轨迹分析积累丰富的先验知识，再起草技能（第 2 节）。(2) **广泛的实验证据**，表明以轨迹为依据的演化所产生的技能能够有效迁移到不同的 LLM 规模、家族和 OOD 任务（第 3 节）。(3) **全面的分析**，揭示整合为何有效：并行合并优于顺序更新；单项整合技能胜过基于检索的推理记忆；智能体诊断能够提升补丁质量；而且补丁价值往往具有组合性（第 4 节）。

2 Trace2Skill

图 2 展示了 Trace2Skill 的三阶段流程：收集轨迹、并行提出轨迹级补丁，再将它们整合为一项可移植技能。我们首先定义技能演化目标，然后描述各个阶段。

2.1 技能与问题形式化

技能是一个人类可读的目录 $\mathcal{S} = (M, \mathcal{R})$，其中 M 是根目录下的 `SKILL.md`，$\mathcal{R}$ 包含辅助参考资料、脚本或资产。根文档存储广泛适用的过程性知识，而辅助文件则提供确定性工具或使用频率较低的细节。令 π_θ 表示推理时使用技能 $\mathcal{S}$ 的固定 LLM 智能体，并令 $\mathcal{P}(\mathcal{S}; \pi_\theta, \mathcal{D})$ 表示该智能体在任务集 $\mathcal{D}$ 上的通过率。给定演化集 $\mathcal{D}_{\text{evolve}}$ 和与之不相交的测试集 $\mathcal{D}_{\text{test}}$，技能演化在不更新 θ 的情况下，根据演化集轨迹构建新技能：

$$\mathcal{S}^* = \mathcal{E}(\mathcal{S}_0, \mathcal{D}_{\text{evolve}}; \pi_\theta), \qquad \mathcal{P}(\mathcal{S}^*; \pi_\theta, \mathcal{D}_{\text{test}}) > \mathcal{P}(\mathcal{S}_0; \pi_\theta, \mathcal{D}_{\text{test}}). \tag{1}$$

我们评估 $\mathcal{S}_0$ 的两种初始化方式：人类专家编写的技能，以及仅凭参数化知识由 LLM 生成的草稿。

2.2 阶段 1：轨迹生成

我们使用 ReAct 风格的运行框架 [Yao et al., 2023]。对于演化集中的每项任务，固定智能体使用 $\mathcal{S}_0$ 运行，并生成一条轨迹 τ_i，其中包含查询、推理与工具使用历史、最终输出以及二元正确性结果。由此得到的语料库 $\mathcal{T}$ 被划分为失败集 $\mathcal{T}^-$ 和成功集 $\mathcal{T}^+$。轨迹生成在演化集问题之间并行进行，因此可以先独立收集轨迹语料库，再提出补丁。提示词模板见 附录 I.1。

2.3 阶段 2：并行补丁提议

一组分析子智能体分别从单条轨迹中独立提出技能补丁。失败轨迹被发送给错误分析器 $\mathcal{A}^-$，成功轨迹被发送给成功分析器 $\mathcal{A}^+$；分析器读取 $\mathcal{S}_0$ 并为其提出补丁，形成补丁池 $\mathcal{P} = \mathcal{P}^- \cup \mathcal{P}^+$。

两种分析器的角色被有意设计为非对称形式。$\mathcal{A}^+$ 采用单遍工作流，从成功轨迹中识别可复用的行为模式。$\mathcal{A}^-$ 采用 ReAct 风格的循环，可以检查轨迹和产物、对照真实答案比较输出，并在提出补丁前验证候选修复方案。无法给出因果解释的失败会被排除，以确保 $\mathcal{P}^-$ 以经过验证的失败机制为依据，而非仅凭日志进行猜测 [Ouyang et al., 2026]。两种角色都会根据技能编写指南，产出简洁且可执行的补丁 [Anthropic, 2026b]。分析器提示词模板和代表性补丁示例见 附录 I.2。

2.4 阶段 3：补丁整合

阶段 3 将补丁池整合为一个连贯的更新 p^*，并将其应用于 $\mathcal{S}_0$。补丁通过 $L = \lceil \log_{B_{\text{merge}}} |\mathcal{P}| \rceil$ 层进行分层合并；在每一层 ℓ，至多将 B_{merge} 个补丁综合为一个补丁：

$$p^{(\ell+1)} = \mathcal{M}\Big(\pi_\theta,\ \mathcal{S}_0,\ \{p_1^{(\ell)}, \ldots, p_{B_{\text{merge}}}^{(\ell)}\}\Big), \qquad \ell = 0, \ldots, L-1, \tag{2}$$

其中，$\mathcal{M}$ 会去重、解决冲突并保留互不重叠的洞见。π_θ 本身同时充当轨迹生成器、分析器和合并算子，因此不需要外部演化模型或教师模型。最终的 p^* 被转换为 diff 风格的编辑，并在确定性防护规则下应用：拒绝对不存在文件的编辑，不应用行范围相冲突的编辑，并验证更新后的技能格式。

合并过程还会执行归纳泛化。由于每个补丁都来自一条轨迹，因此独立补丁中反复出现的编辑表明存在系统性的任务属性，而非一次性修复。因此，我们要求 $\mathcal{M}$ 优先选择普遍存在的模式，并丢弃仅适用于个别情况的编辑，从而产出一个精炼技能来替换 $\mathcal{S}_0$，且该技能无需检索即可直接用于推理。合并算子的提示词模板及整合补丁 p^* 的示例见 附录 I.3。

Trace2Skill 支持两种模式，覆盖有无专家编写技能的场景：技能深化从人工编写的技能开始，技能创建则从依据 LLM 参数化知识生成的技能开始。两者都使用从轨迹分析中归纳出的补丁来改进目标技能。

3 实验

3.1 实验设置

电子表格设置。 我们的主要实验采用 SpreadsheetBench-Verified [Ma et al., 2024]，其中智能体通过使用工具来操作 `xlsx` 文件。我们将其 400 个样本划分为 200 个演化问题和 200 个留出测试问题，同时还报告完整 SpreadsheetBench（排除演化集中的所有问题）上的 Soft/Hard 分数，以及向转换为电子表格格式的 WikiTableQuestions [Pasupat and Liang, 2015] 和 HiTab [Cheng et al., 2022] 的 OOD 迁移结果。所有电子表格结果均使用各基准的官方评估标准，并对三个随机种子的结果取平均。

技能设置。 我们比较 **No Skill**、Anthropic 的 `xlsx` 技能（**Human-Written**）、使用 **Parametric** 知识生成的 LLM 技能，以及三种 Trace2Skill 变体：**+Error**、**+Success** 和 **+Combined**，它们分别只整合失败轨迹中的补丁、只整合成功轨迹中的补丁，以及整合所有轨迹中的补丁。*Skill Deepening* 从 Human-Written 出发，而 *Skill Creation* 从 Parametric 出发。我们同时将 Qwen3.5-122B-A10B 和 Qwen3.5-35B-A3B 作为技能编写模型与技能使用模型进行评估。我们采用 100% 自演化：由同一个模型生成轨迹、提出补丁并编辑技能。数据集构建、评分和模型服务的详细信息见 附录 A；外部 `skill-creator` 基线见 附录 H。

3.2 主要结果

表 1 报告了不同技能条件、编写模型、使用模型和迁移任务下的全部电子表格结果。基线行给出绝对分数；演化后的行给出相对于对应基线的带符号差值，其中 Deepening 与 Human-Written 比较，Creation 与 Parametric 比较。我们将 **Avg** 作为主要汇总指标，因为它对两个使用模型上的 ID 与 OOD 性能赋予相同权重，从而奖励能够跨模型、跨任务泛化的技能。

基线揭示了两种设置为何都很重要。 Human-Written 是一种有用的手工先验，但无法在不同模型规模之间稳定迁移；Parametric 的表现仍接近 No Skill，这证实仅靠参数化知识无法产生可执行的电子表格技能 [Han et al., 2026]。这些基线引出了两种评估范式：Deepening 检验能否让一个强大的人工技能具备更好的可迁移性，而 Creation 检验基于轨迹的蒸馏能否从较弱的种子构建出有用的技能。

Deepening 和 Creation 都能产生可泛化的技能。 从 Human-Written 出发，演化后的技能持续提升分布内电子表格任务的性能，并且通常能够迁移到其他模型规模和 OOD 表格任务；从 Parametric 出发，在有利设置下，Creation 可以达到或超过 Human-Written 的质量。这些收益十分广泛：它们出现在不同的编写模型、使用模型和任务族中，而非仅出现在生成轨迹的模型上。值得注意的是，35B Deepening +Combined 取得了最佳的绝对 Avg，这表明由小型 LLM 编写的技能同样可以实现泛化。在不同分析器中，+Error 和 +Success 都能改善电子表格任务，且 +Combined 通常带来最大的 Avg 提升。

表 1: 不同技能编写模型（负责演化技能）和技能使用模型（在推理时运行技能）组合上的电子表格主要结果。SpreadsheetBench 报告 Vrf（验证分数）、Soft（问题通过率）和 Hard（任务通过率）；WikiTQ 与 HiTab 是分布外（OOD）表格问答迁移任务，Avg=$(\text{ID}_{\text{avg}}+\text{OOD}_{\text{avg}})/2$ 对分布内（ID，SpreadsheetBench）和 OOD 指标赋予相同权重。参考行给出绝对分数；演化后各行给出相对于人工编写（技能深化）或参数化知识（技能创建）基线的带符号差值。每列最大的绝对分数以**粗体**标出。各随机种子的标准差见 表 7。

	技能使用模型：*Qwen3.5-122B-A10B*					技能使用模型：*Qwen3.5-35B-A3B*					
	SpreadsheetBench			*OOD*		*SpreadsheetBench*			*OOD*		
条件	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**HiTab↑**	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**HiTab↑**	**Avg↑**
基线（绝对分数）											
无技能	27.67	28.90	17.57	21.50	14.42	19.00	18.00	4.60	13.33	19.12	18.19
人工编写	48.33	36.30	17.03	74.68	41.31	9.67	13.03	3.37	9.02	5.30	26.93
参数化知识	26.17	36.60	17.50	23.73	17.36	20.17	13.70	3.87	20.14	13.94	19.23
技能编写模型：*Qwen3.5-122B-A10B*											
技能深化（相对初始人工编写技能的差值）											
+Error	+17.50	+10.30	+10.40	+1.62	+2.14	**+27.00**	+9.44	+2.86	+9.26	+9.55	+9.28
+Success	+18.00	+8.60	+8.70	-10.35	-0.95	+19.66	+6.84	+1.33	+12.09	+22.33	+8.15
+Combined	**+21.50**	**+10.87**	**+12.50**	+4.56	+2.97	+21.16	+8.84	+1.80	+6.64	+11.94	+9.65
技能创建（相对初始参数化知识技能的差值）											
+Error	+22.83	+3.77	+5.87	+7.89	-1.70	+8.66	+9.53	+4.00	+2.06	+9.64	+6.79
+Success	+15.33	-0.93	+4.33	+23.70	+19.89	+12.83	+11.57	+6.13	+30.36	**+28.90**	+16.96
+Combined	+14.00	-0.63	+3.53	+32.32	+23.11	+15.50	**+14.50**	**+7.23**	+29.70	+27.71	+18.62
技能编写模型：*Qwen3.5-35B-A3B*											
技能深化（相对初始人工编写技能的差值）											
+Error	+16.67	+8.50	+8.14	-6.36	-2.38	+17.33	+9.17	+4.83	+2.71	+8.08	+5.64
+Success	+2.17	+2.73	+3.30	+1.46	+1.70	+12.33	+5.87	+1.23	**+43.23**	+34.70	+12.44
+Combined	+6.67	+3.87	+4.17	+2.65	+2.44	+20.00	+5.77	+2.36	+42.20	+36.46	**+14.04**
技能创建（相对初始参数化知识技能的差值）											
+Error	+1.00	-7.70	+1.03	**+57.65**	**+28.25**	+3.83	+7.30	+2.66	+12.66	+17.81	+15.22
+Success	+5.33	-4.57	+2.43	+9.09	+2.34	+5.66	+5.80	+2.63	+3.31	+18.94	+5.65
+Combined	+8.33	-5.83	+2.00	+30.82	+16.93	+4.33	+9.73	+4.73	+18.00	+25.22	+13.31

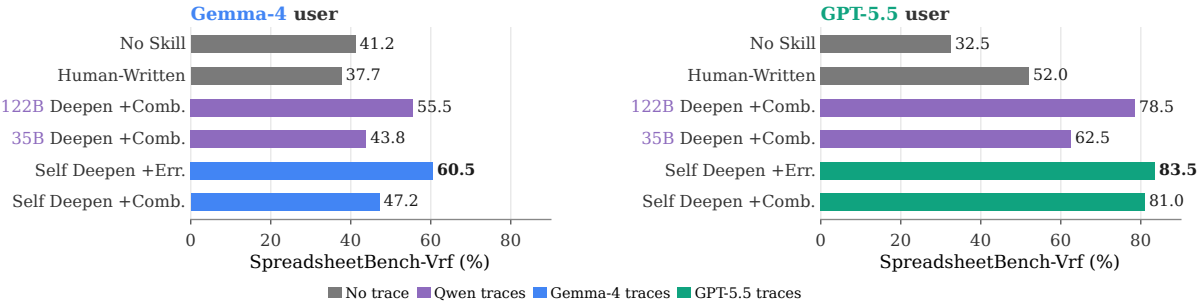


图 3: 在 SpreadsheetBench 上的跨模型族泛化。Gemma-4-31B-it 和 GPT-5.5-high 分别（i）基于各自的轨迹演化技能，以及（ii）运行由 Qwen3.5 编写的技能。

3.3 跨模型族泛化

我们使用 Gemma-4-31B-it 和 GPT-5.5-high，在两种设置下测试跨模型族泛化：每个模型（i）基于自身轨迹深化人工编写的 `xlsx` 技能，所用方法为 Trace2Skill；以及（ii）运行 Qwen3.5-122B/35B +Combined 深化后的技能。图 3 表明，两个模型都能从自身轨迹中实现自我提升，也都能受益于 Qwen3.5 编写的技能，因此 Trace2Skill 能够跨模型族泛化。附录 B.4 报告了更多结果和实现细节。

3.4 数学推理

我们将 Trace2Skill 应用于数学领域，以检验其领域无关性。与电子表格设置（第 3.1 节）相同，数学智能体在 ReAct 循环中运行，并使用命令行 Python 解释器为每个问题编写和执行代码。我们在 400 道 DAPO 题目

表 2: 数学技能在留出的 DAPO 与 AIME 上的迁移结果。D-Test 表示 DAPO-Math-Test 通过率；AIME 使用 avg@8。

条件	技能使用模型：*122B* D-Test↑	AIME↑	技能使用模型：*35B* D-Test↑	AIME↑
参考（绝对分数）				
无技能	91.0	90.4	89.0	83.3
技能编写模型：*Qwen3.5-122B-A10B*				
技能创建（初始设置：无技能）				
+Error	**+4.0**	**+2.9**	+5.0	+5.0
+Success	+2.0	+0.4	+2.0	+4.6
+Combined	+3.0	-1.2	**+6.0**	**+5.5**
技能编写模型：*Qwen3.5-35B-A3B*				
技能创建（初始设置：无技能）				
+Error	+3.0	+1.3	+4.0	+0.5
+Success	-1.0	-1.2	+0.0	-1.2
+Combined	+1.0	-0.4	+1.0	+0.0

表 3: 由轨迹蒸馏得到的视觉推理技能在 DocVQA 上的迁移结果。ANLS 表示相似度；Acc 表示 ANLS ≥ 0.5 的比例。

条件	技能使用模型：*122B* ANLS↑	Acc↑	技能使用模型：*35B* ANLS↑	Acc↑
参考（绝对分数）				
无技能	0.6300	70.27	0.6582	73.17
技能编写模型：*Qwen3.5-122B-A10B*				
技能创建（初始设置：无技能）				
+Error	+0.1949	+17.69	+0.1974	+16.64
+Success	+0.1639	+14.89	+0.1668	+14.23
+Combined	**+0.2534**	**+22.25**	+0.2049	+17.22
技能编写模型：*Qwen3.5-35B-A3B*				
技能创建（初始设置：无技能）				
+Error	+0.0884	+6.79	+0.1267	+10.15
+Success	+0.1572	+14.53	+0.2132	+18.27
+Combined	+0.0958	+7.73	**+0.2158**	**+18.83**

[Yu et al., 2025] 上从零创建技能，并按照 第 3.2 节 的跨模型协议，在 100 道互不重叠的留出 DAPO 题目（ID）和 AIME 2026（OOD 竞赛数学；avg@8，基于 30 道题计算）上进行评估。 表 2 报告相对于 No Skill 的差值。**Trace2Skill 适用于数学领域：**蒸馏得到的技能同时改善了留出 DAPO 和 OOD AIME 上的表现，而非过拟合源分布。 **+Error 是最稳定的信号**，能够在 122B 和 35B 使用模型之间顺利迁移。

3.5 视觉问答

为测试多模态泛化能力，我们将 Trace2Skill 应用于 DocVQA [Mathew et al., 2020]；在该任务中，智能体需要回答有关表单、表格、发票、信件和报告等文档图像的问题。同样，每个智能体都在配有命令行 + Python 环境的 ReAct 循环中运行。我们仅使用 50 个 DocVQA 样例进行技能演化，并将其从评估中移除；其余 5,299 个样例构成留出测试集。我们报告官方 ANLS 和 Accuracy（ANLS ≥ 0.5，%），涵盖同模型使用，以及由 122B 和 35B 编写的技能之间的跨模型迁移。表 3 中的结果表明，**所有演化后的 DocVQA 技能都比 No Skill 表现更好。+Combined 是最强的设置**，它带来了最明显的同模型收益和正向跨模型迁移。

4 分析

本节分析 Trace2Skill 为何有效及其更广泛的应用。我们首先在共享轨迹池和执行框架下分别考察其三项核心设计选择（第 4.1 节），随后刻画其蒸馏出的标准操作程序（第 4.2 节），研究补丁价值的组合方式（第 4.3 节），并在更广泛的真实世界任务中测试迁移能力（第 4.4 节）。

4.1 核心设计比较

Trace2Skill 建立在三项设计选择之上。我们在相同的轨迹池和执行框架下，将每一项选择与其自然替代方案进行比较，以分别考察其效果：并行多对一整合与顺序编辑（在不损失质量的前提下提速）、单一整合技能与经验记忆检索（无需测试时检索即可复用），以及基于智能体的错误分析与单次调用错误分析（得到更准确的根因补丁）。

表 4: SpreadsheetBench 上的并行整合与顺序编辑比较（使用相同轨迹池，+Error）。并行方法在效果和效率上均优于顺序方法。表 8 和 表 10 表明数学/VQA 任务也呈现相似趋势。

条件	技能使用模型：*122B* Vrf↑	Soft↑	Hard↑	技能使用模型：*35B* Vrf↑	Soft↑	Hard↑	时间↓
顺序-B=4	59.00	40.63	20.63	26.17	22.37	7.47	∼15 分钟
顺序-B=1	61.83	44.40	25.40	26.00	**23.83**	**10.57**	∼60 分钟
并行（本文）	**65.83**	**46.60**	**27.43**	**27.00**	22.20	8.20	**∼3 分钟**

并行整合。 在线技能演化通常会随着轨迹批次的到来而编辑技能，因此后续分析依赖于先前的编辑。我们通过与两个顺序基线比较，单独考察并行多对一整合的效果：Seq-B=1 在每条轨迹后更新，Seq-B=4 在每四条

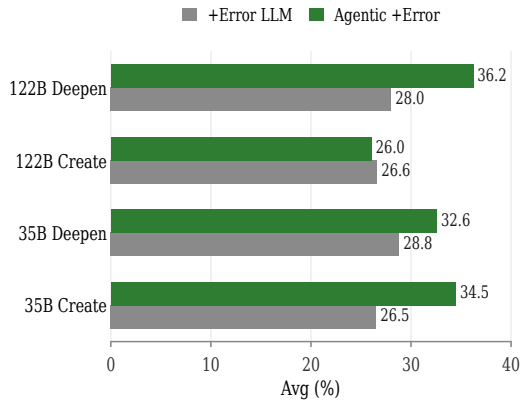


图 4: 基于智能体的 +Error 与单次调用 +Error LLM 在 Avg 上的比较；Avg 平衡了 ID 与 OOD 性能（与表 1 中的 Avg 相同）。基于智能体的分析器会检查产物并验证修复，而 +Error LLM 只读取执行日志。

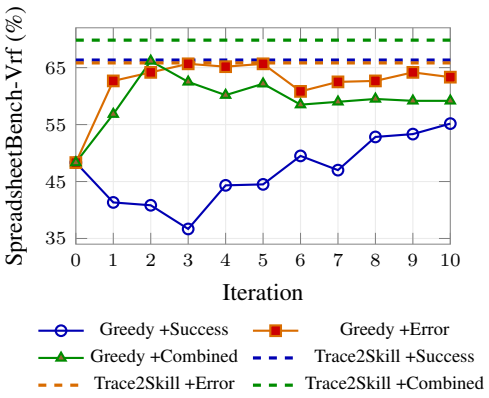


图 5: SpreadsheetBench-Vrf 上的贪心选择性补丁聚合（通过率，%）。每条曲线在每次迭代（x 轴）中从 Error、Success 或 Combined 补丁池加入一个通过验证选出的补丁；水平线表示 Trace2Skill 对全部补丁的完整聚合。

轨迹后更新。所有条件均仅使用错误分析器，并从 Human-Written 技能初始化。 表 4 报告了由此产生的质量与墙钟时间权衡。

并行整合在 122B 的所有 SpreadsheetBench 指标以及 35B Vrf 上表现更好；唯一的例外是 Seq-B=1 在 35B Soft 和 Hard 上略高。这一较小的质量权衡对应着巨大的效率差距：并行方法约用 3 min 即可生成技能，而其余两者分别需要 15 min（Seq-B=4）和 60 min（Seq-B=1）。数学推理和 DocVQA 也呈现相同的总体趋势：并行整合在保持或提升质量的同时速度快得多；附录 B 给出了延迟分析，以及数学/VQA 上的类似比较。

整体技能与检索。 历史经验检索是复用智能体轨迹的一种常见范式：经验记忆系统存储以往执行过程中的反思、记忆或程序性经验，并在推理时检索相关条目 [Shinn et al., 2023, Wang et al., 2023a, Ouyang et al., 2026, Fang et al., 2026, Wang et al., 2024b, Liu et al., 2025]。

我们使用 ReasoningBank [Ouyang et al., 2026] 实例化这一范式：遵循其原始设置，我们存储来自成功和失败轨迹的经验，并检索 top-1 记忆，所用模型为 Qwen3-Embedding-8B。我们将其与使用相同轨迹池的 +Combined 进行比较。同模型 Deepening 的结果如表 5 所示。+Combined 始终优于 ReasoningBank。这说明，将轨迹证据整合为紧凑技能比在测试时检索彼此孤立的记忆更具优势。我们排除了 OOD 评估，因为 WikiTQ 和 HiTab 的查询在语义上与 Spreadsheet-Bench 的查询相距太远，不适合应用检索。

表 5: 在 SpreadsheetBench 上，Trace2Skill 优于 ReasoningBank 式检索（使用相同轨迹池）。表 9 和 表 11 表明数学/VQA 任务也呈现相似趋势。

设置	使用模型	Vrf↑	Soft↑	Hard↑
ReasoningBank [Ouyang et al., 2026]	122B	56.00	40.10	21.30
	35B	20.50	17.30	4.97
人工编写 +Combined（本文）	122B	**69.83**	**47.17**	**29.53**
	35B	**29.67**	**18.80**	**5.73**

基于智能体的错误分析。 相关工作通过单次非交互式 LLM 调用，从错误轨迹中提炼可迁移的经验或技能 [Ouyang et al., 2026, Xia et al., 2026a, Yang et al., 2026, Jiang et al., 2026]。我们使用 **+Error LLM** 对这一设计进行消融：一次 LLM 调用读取每条失败轨迹并提出补丁，但不检查产物、不查询真实答案，也不验证修复；图 4 将其与我们基于智能体的错误分析器进行比较。在大多数设置中，基于智能体的 +Error 的 Avg 高于 +Error LLM，这表明交互式诊断比仅基于日志生成补丁更有用；附录 D 报告并讨论了 表 12 中按数据集划分的完整结果。附录 C.1 中的定性分析揭示了其机制：访问产物并验证修复有助于基于智能体的分析器更准确地识别根因。

同等条件比较与系统正面对比。 上述三项比较均在同等条件下进行，在不受数据、模型、执行框架和工程因素这些会使全系统评估复杂化的混杂影响下，分别考察各项核心设计。作为补充视角，附录 G 报告了在共享开放模型和基准下，与三个同期系统（XSkill [Jiang et al., 2026]、EvoSkill [Alzubi et al., 2026] 和 SkillGen [Ma et al., 2026]）的直接正面对比，其中 Trace2Skill 仍展现出优势。

4.2 学到的 SoP

多对一合并会保留跨轨迹反复出现的操作，并将其转化为标准操作程序（SoP），而不是收集一次性技巧。在 323 个 Map 阶段补丁（来自 122B Deepening +Combined 运行）中，有四种学到的 SoP 占据主导地位，每种都被 16% 至 55% 的补丁引用（一条补丁可以引用多个 SoP），包括：写入后重新计算并回读公式、提交前核验目标单元格，以及按不会导致文件损坏的顺序删除行。特定于任务的特殊情况会被转入按需使用的 `references/` 文件，使 SKILL.md 专注于可广泛复用的流程（详见 附录 C.3）。

表 6: 在 122B Deepening 电子表格设置中，BO 选择的 +Error 补丁与应用全部补丁（不筛选）的比较。“演化集”列表示在演化集上的准确率。

技能	演化集	Vrf	Soft	Hard	WikiTQ	HiTab
不筛选（+Error）	68.00	65.83	46.60	27.43	76.30	**43.45**
BO 筛选（+Error）	**72.83**	**69.83**	**47.27**	**28.90**	**77.51**	43.38

4.3 选择性补丁聚合

Trace2Skill 将所有学到的补丁聚合进目标技能，从而避免逐补丁验证的成本。如果补丁质量参差不齐，选择其中一个子集能否取得更好的效果？我们测试了两种选择器，并与完整聚合进行比较：**贪心 top-1** 在每次迭代中，根据演化集上的验证结果加入局部最优补丁；**贝叶斯优化（BO）** 搜索二元补丁纳入向量，并复用先前尝试过的子集的验证分数，使后续提案更偏向有前景的组合。两者都使用包含 200 个任务的演化集（第 3.2 节），以及从演化集中抽取的 32 个任务的验证集；附录 E 给出了完整的算法细节。

贪心方法上升很快，但在低于完整聚合的位置进入平台期。 图 5 显示，贪心 +Error 和 +Combined 在前几次迭代中上升，随后进入平台期；+Success 先下降，之后仅部分恢复；所有贪心曲线都低于 Trace2Skill 的完整聚合。平台期由两种机制导致（详见 附录 C.2）。 (1) 补丁无关回退：补丁使部分目标任务从错误变为正确，但其副作用也使原本正确的任务从正确变为错误，因此净准确率停滞不前。 (2) 语义重叠：反复出现的错误会使流水线提出反复针对相同行为的补丁（例如公式重算、验证检查清单），因此新补丁大多只是重述已有指导，带来的边际收益很小。因此，补丁价值取决于组合，而非单个补丁价值的简单相加，这使得优化补丁的组合效果成为必要。

BO 有效，但计算成本高。 BO 直接搜索补丁子集，因此能够对贪心方法遗漏的互补性和干扰进行建模。表 6 表明，与应用全部 +Error 补丁相比，BO 改善了大多数所选指标，尤其是 SpreadsheetBench 各列。其代价在于验证：每个候选子集都必须被实例化为技能，并在验证集上运行，之后才能为后验分布提供信息；补丁全集越大，这一成本增长得越快。因此，我们认为 BO 适用于目标分布和验证预算已知的场景，而不应取代 Trace2Skill 中默认的完整聚合。

4.4 更广泛的应用

我们在 PDF 提取、PPTX 编辑和 DOCX 编辑中测试 Trace2Skill 的更广泛应用。这些设置以 Anthropic 官方的 `pdf`、`pptx` 和 `docx` 技能为起点 [Anthropic, 2026a]，并沿用电子表格实验的执行方式：智能体使用命令行工具操作文件，并由任务专用的本地验证器评分。

在这三个领域中，基于源轨迹的演化都提升了独立留出目标上的性能。对于 PDF，VRDU 轨迹可迁移到 VAREX，将通过率从 76.9% 提升到 85.3% [Wang et al., 2023b, Barzelay et al., 2026]。对于 PPTX，TSBench 轨迹可迁移到演示文稿互不重叠的 TSBench OOD 划分，将性能从 72.5% 提升到 88.8% [Jung et al., 2026]。对于 DOCX，在覆盖通用 DOCX 操作的合成任务上演化，能够迁移到 OfficeBench DOCX 子集，将性能从 79.7% 提升到 87.5% [Wang et al., 2024a]。附录 F 给出了各领域的设置和实现细节。

5 相关工作

智能体技能。 智能体技能将任务流程、领域知识和操作防护规则封装为可加载的产物 [Anthropic, 2026d]，但技能生态和基准表明，这一抽象需要谨慎设计：聚焦且匹配良好的技能可以提升性能，而宽泛、过时或不匹配的技能则可能分散智能体的注意力，甚至损害其表现 [Zhou et al., 2026b, Li et al., 2026b, Han et al., 2026, Li, 2026, Li et al., 2026a, Liang et al., 2026]。Trace2Skill 延续了将技能视为可移植 SoPs 的观点，但提出了一个更聚焦的问题：如何将大量执行证据压缩为在不同模型和任务分布中始终有用的指南。

用于智能体自我演化的经验记忆。 另一类工作通过存储执行经验以供复用来改进智能体，其形式从语言反思和累积行为，到在测试时查询的检索记忆、过程记忆、工作流记忆或回放记忆不等 [Shinn et al., 2023, Wang et al., 2023a, Ouyang et al., 2026, Fang et al., 2026, Wang et al., 2024b, Qian et al., 2024, Nottingham et al., 2024, Liu et al., 2025]。这些方法与我们一样，都以“经验中包含可复用结构”为前提，但它们保留了外部记忆或检索模块，而 Trace2Skill 将许多局部观察蒸馏到一个静态技能目录中。它更倾向于归纳压缩而非最近邻复用，并避免在测试时依赖检索质量。

技能与策略演化。 与我们最接近的同期工作从智能体交互或引导式优化中演化技能 [Zheng et al., 2025, Yang et al., 2026, Jiang et al., 2026, Alzubi et al., 2026, Zhou et al., 2026a, Ma et al., 2026, Anthropic, 2026b]，而更广泛的一类工作则通过技能增强强化学习、内在技能演化或元学习，在线协同演化策略、技能或智能体循环 [Xia et al., 2026a, Li et al., 2026c, Xia et al., 2026b]。 Trace2Skill 提出了一个互补的问题：能否将使用一个模型收集的轨迹压缩到静态技能目录中，从而直接使其他模型和任务受益。它不维护测试时记忆、检索索引或更新后的

策略，而是将轨迹局部证据整合为可复用的 SoPs，并可原样加载。因此，我们关注的不是特定智能体循环中的自适应复用，而是所学产物在不同部署环境中的可移植性。

6 结论

我们提出了 Trace2Skill，这是一个将智能体执行轨迹蒸馏为可移植技能的框架：并行的分析子智能体从互不相交的轨迹批次中提出有针对性的补丁，再由整合步骤一次性将它们合并到一个声明式技能目录中。从单一模型的轨迹中蒸馏出的技能可以跨模型规模、家族和 OOD 任务迁移。Trace2Skill 在多个领域也展现出很强的适用性。

局限性

Trace2Skill 默认应用所有整合后的补丁；使用贝叶斯优化（BO）选择质量更高的子集可能有所帮助（第 4.3 节），但成本很高。可靠的选择信号需要一个大型且有代表性的验证集，而且 BO 必须为每个候选子集实例化一项新技能并进行评分，因此成本会随着验证集和补丁全集的扩大而迅速增长。理想情况下，使用整个演化集进行验证，比我们仅包含 32 个问题的采样验证更能反映补丁质量，但这会带来高得多的计算成本。因此，我们将对组合式补丁选择的更全面探索留待未来工作。

伦理声明

我们使用公开可用的数据集，不存在数据隐私问题。我们使用的所有产物均采用允许研究用途的许可证。定性分析中的人工标注由本文作者完成。我们未发现本研究还存在其他伦理风险。我们使用 AI 编程工具（Codex 和 Claude Code）辅助编码，并确认所有此类编码工作都经过严格的人工监督。我们实现了单元测试，以避免钻空子或非对齐行为。我们还将开源代码，以实现完整的可复现性。此外，我们也使用 ChatGPT 检查并修正语法，整个过程均由人类作者全面监督。

References

Salaheddin Alzubi, Noah Provenzano, Jaydon Bingham, Weiyuan Chen, and Tu Vu. Evoskill: Automated skill discovery for multi-agent systems, 2026. URL https://arxiv.org/abs/2603.02766.

Anthropic. PDF, DOCX, and PPTX document skills. GitHub repository, 2026a. URL https://github.com/anthropics/skills/tree/main/skills. Accessed: 2026-05-01; document-skill subfolders: `pdf`, `docx`, and `pptx`.

Anthropic. How to create a skill with claude through conversation. Claude Tutorials, 2026b. URL https://claude.com/resources/tutorials/how-to-create-a-skill-with-claude-through-conversation.

Anthropic. skill-creator. GitHub repository, 2026c. URL https://github.com/anthropics/skills/tree/main/skills/skill-creator.

Anthropic. What are skills? Claude Help Center, 2026d. URL https://support.claude.com/en/articles/12512176-what-are-skills. Access Date: 2026-03-22.

Udi Barzelay, Ophir Azulai, Inbar Shapira, Idan Friedman, Foad Abo Dahood, Madison Lee, and Abraham Daniels. VAREX: A benchmark for multi-modal structured extraction from documents, 2026. URL https://arxiv.org/abs/2603.15118.

Zhoujun Cheng, Haoyu Dong, Zhiruo Wang, Ran Jia, Jiaqi Guo, Yan Gao, Shi Han, Jian-Guang Lou, and Dongmei Zhang. HiTab: A hierarchical table dataset for question answering and natural language generation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1094–1110, Dublin, Ireland, May 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.acl-long.78. URL https://aclanthology.org/2022.acl-long.78/.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2026. URL https://arxiv.org/abs/2508.06433.

Google. Gemma 4: Byte for byte, the most capable open models. https://blog.google/innovation-and-ai/technology/developers-tools/gemma-4/, 2026. Accessed: 2026-05-18.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. Swe-skills-bench: Do agent skills actually help in real-world software engineering?, 2026. URL https://arxiv.org/abs/2603.15401.

Guanyu Jiang, Zhaochen Su, Xiaoye Qu, and Yi R. Fung. Xskill: Continual learning from experience and skills in multimodal agents, 2026. URL https://arxiv.org/abs/2603.12056.

Kyudan Jung, Hojun Cho, Jooyeol Yun, Soyoung Yang, Jaehyeok Jang, and Jaegul Choo. Talk to your slides: High-efficiency slide editing via language-driven structured data manipulation, 2026. URL https://arxiv.org/abs/2505.11604.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL https://arxiv.org/abs/2309.06180.

Hao Li, Chunjiang Mu, Jianhao Chen, Siyue Ren, Zhiyao Cui, Yiqun Zhang, Lei Bai, and Shuyue Hu. Organizing, orchestrating, and benchmarking agent skills at ecosystem scale, 2026a. URL https://arxiv.org/abs/2603.02176.

Jiachun Li, Pengfei Cao, Zhuoran Jin, Yubo Chen, Kang Liu, and Jun Zhao. Mirage: Evaluating and explaining inductive reasoning process in language models, 2025. URL https://arxiv.org/abs/2410.09542.

Xiangyi Li, Wenbo Chen, Yimin Liu, Shenghan Zheng, Xiaokun Chen, Yifeng He, Yubo Li, Bingran You, Haotian Shen, Jiankai Sun, Shuyi Wang, Binxu Li, Qunhong Zeng, Di Wang, Xuandong Zhao, Yuanli Wang, Roey Ben Chaim, Zonglin Di, Yipeng Gao, Junwei He, Yizhuo He, Liqiang Jing, Luyang Kong, Xin Lan, Jiachen Li, Songlin Li, Yijiang Li, Yueqian Lin, Xinyi Liu, Xuanqing Liu, Haoran Lyu, Ze Ma, Bowei Wang, Runhui Wang, Tianyu Wang, Wengao Ye, Yue Zhang, Hanwen Xing, Yiqi Xue, Steven Dillmann, and Han chung Lee. Skillsbench: Benchmarking how well agent skills work across diverse tasks, 2026b. URL https://arxiv.org/abs/2602.12670.

Xiaoxiao Li. When single-agent with skills replace multi-agent systems and when they fail, 2026. URL https://arxiv.org/abs/2601.04748.

Yu Li, Rui Miao, Zhengling Qi, and Tian Lan. Arise: Agent reasoning with intrinsic skill evolution in hierarchical reinforcement learning, 2026c. URL https://arxiv.org/abs/2603.16060.

Yuan Liang, Ruobin Zhong, Haoming Xu, Chen Jiang, Yi Zhong, Runnan Fang, Jia-Chen Gu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Xin Xu, Tongtong Wu, Kun Wang, Yang Liu, Zhen Bi, Jungang Lou, Yuchen Eleanor Jiang, Hangcheng Zhu, Gang Yu, Haiwen Hong, Longtao Huang, Hui Xue, Chenxi Wang, Yijun Wang, Zifei Shan, Xi Chen, Zhaopeng Tu, Feiyu Xiong, Xin Xie, Peng Zhang, Zhengke Gui, Lei Liang, Jun Zhou, Chiyu Wu, Jin Shang, Yu Gong, Junyu Lin, Changliang Xu, Hongjie Deng, Wen Zhang, Keyan Ding, Qiang Zhang, Fei Huang, Ningyu Zhang, Jeff Z. Pan, Guilin Qi, Haofen Wang, and Huajun Chen. Skillnet: Create, evaluate, and connect ai skills, 2026. URL https://arxiv.org/abs/2603.04448.

Brian S. Lin, Jiaxin Yuan, Zihan Zhou, Shouli Wang, Shuo Wang, Cunliang Kong, Qi Shi, Yuxuan Li, Liner Yang, Zhiyuan Liu, and Maosong Sun. On llm-based scientific inductive reasoning beyond equations, 2025. URL https://arxiv.org/abs/2509.16226.

Yitao Liu, Chenglei Si, Karthik Narasimhan, and Shunyu Yao. Contextual experience replay for self-improvement of language agents, 2025. URL https://arxiv.org/abs/2506.06698.

Yuchen Ma, Yue Huang, Han Bao, Haomin Zhuang, Swadheen Shukla, Michel Galley, Xiangliang Zhang, and Stefan Feuerriegel. Skillgen: Verified inference-time agent skill synthesis, 2026. URL https://arxiv.org/abs/2605.10999.

Zeyao Ma, Bohan Zhang, Jing Zhang, Jifan Yu, Xiaokang Zhang, Xiaohan Zhang, Sijia Luo, Xi Wang, and Jie Tang. Spreadsheetbench: Towards challenging real world spreadsheet manipulation, 2024. URL https://arxiv.org/abs/2406.14991.

Minesh Mathew, Dimosthenis Karatzas, R Manmatha, and CV Jawahar. Docvqa: A dataset for vqa on document images. corr abs/2007.00398 (2020). *arXiv preprint arXiv:2007.00398*, 2020.

Kolby Nottingham, Bodhisattwa Prasad Majumder, Bhavana Dalvi Mishra, Sameer Singh, Peter Clark, and Roy Fox. Skill set optimization: Reinforcing language model behavior via transferable skills, 2024. URL https://arxiv.org/abs/2402.03244.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. Reasoningbank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=jL7fwchScm.

Panupong Pasupat and Percy Liang. Compositional semantic parsing on semi-structured tables, 2015. URL https://arxiv.org/abs/1508.00305.

Cheng Qian, Shihao Liang, Yujia Qin, Yining Ye, Xin Cong, Yankai Lin, Yesai Wu, Zhiyuan Liu, and Maosong Sun. Investigate-consolidate-exploit: A general strategy for inter-task agent self-evolution, 2024. URL https://arxiv.org/abs/2401.13996.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Qwen Team. Qwen3.5: Accelerating productivity with native multimodal agents, February 2026. URL https://qwen.ai/blog?id=qwen3.5.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models, 2023a. URL https://arxiv.org/abs/2305.16291.

Zilong Wang, Yichao Zhou, Wei Wei, Chen-Yu Lee, and Sandeep Tata. VRDU: A benchmark for visually-rich document understanding. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2023b. doi: 10.1145/3580305.3599929. URL https://arxiv.org/abs/2211.15421.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. OfficeBench: Benchmarking language agents across multiple applications for office automation, 2024a. URL https://arxiv.org/abs/2407.19056.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2024b. URL https://arxiv.org/abs/2409.07429.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Skillrl: Evolving agents via recursive skill-augmented reinforcement learning, 2026a. URL https://arxiv.org/abs/2602.08234.

Peng Xia, Jianwen Chen, Xinyu Yang, Haoqin Tu, Jiaqi Liu, Kaiwen Xiong, Siwei Han, Shi Qiu, Haonian Ji, Yuyin Zhou, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. Metaclaw: Just talk – an agent that meta-learns and evolves in the wild, 2026b. URL https://arxiv.org/abs/2603.17187.

Chenfei Xiong, Jingwei Ni, Yu Fan, Vilém Zouhar, Donya Rooein, Lorena Calvo-Bartolomé, Alexander Hoyle, Zhijing Jin, Mrinmaya Sachan, Markus Leippold, Dirk Hovy, Mennatallah El-Assady, and Elliott Ash. Co-DETECT: Collaborative discovery of edge cases in text classification. In Ivan Habernal, Peter Schulam, and Jörg Tiedemann, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 354–364, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-334-0. doi: 10.18653/v1/2025.emnlp-demos.25. URL https://aclanthology.org/2025.emnlp-demos.25/.

Yutao Yang, Junsong Li, Qianjun Pan, Bihao Zhan, Yuxuan Cai, Lin Du, Jie Zhou, Kai Chen, Qin Chen, Xin Li, Bo Zhang, and Liang He. Autoskill: Experience-driven lifelong learning via skill self-evolution, 2026. URL https://arxiv.org/abs/2603.01145.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL https://arxiv.org/abs/2210.03629.

Qiying Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL https://arxiv.org/abs/2503.14476.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. Skillweaver: Web agents can self-improve by discovering and honing skills, 2025. URL https://arxiv.org/abs/2504.07079.

表 7: 表 1 中电子表格主要结果在随机种子 41、42 和 43 上的标准差。参考行给出绝对分数的标准差；演化后各行给出 表 1 所定义的配对差值的标准差。汇总指标 Avg 十分稳定（每行标准差均为 ≤ 3.4），且 表 1 中的平均增益相对于各单元格的波动仍然较大，因此比较结果稳健。个别列（Vrf 以及 OOD 的 WikiTQ/HiTab 迁移）方差较高属于预期现象，并不表示不稳定：每个分数都汇总了较长的智能体运行轨迹，环境交互往往超过 30 轮，因此单个基准上的逐次运行随机性会累积，但在 Avg 中会被平均。为简洁起见，本文省略其他结果的标准差表；这些结果的标准差同样较小，电子表格任务的标准差范围与本表相近。DAPO-Test、AIME'26 和 DocVQA ANLS 的标准差范围分别为 1.1–2.4、1.8–3.4 和 0.034–0.139。

	技能使用模型：*Qwen3.5-122B-A10B*					技能使用模型：*Qwen3.5-35B-A3B*					
	SpreadsheetBench			*OOD*		*SpreadsheetBench*			*OOD*		
条件	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**HiTab↑**	**Vrf↑**	**Soft↑**	**Hard↑**	**WikiTQ↑**	**HiTab↑**	**Avg↑**
基线（绝对分数）											
无技能	3.75	0.90	0.70	0.81	0.67	1.80	2.77	1.25	1.27	5.28	0.42
人工编写	8.10	1.39	0.40	0.65	0.71	2.93	0.57	0.40	0.36	2.68	0.46
参数化知识	0.58	1.04	0.75	6.52	8.93	1.44	1.56	0.50	13.32	18.31	3.28
技能编写模型：*Qwen3.5-122B-A10B*											
技能深化（相对初始人工编写技能的差值）											
+Error	4.82	1.10	0.78	0.78	0.60	6.76	3.36	2.06	2.62	4.16	1.02
+Success	12.53	0.75	1.35	1.68	0.75	3.69	3.78	2.23	1.43	4.01	1.43
+Combined	9.53	1.68	0.61	1.22	0.85	5.92	2.92	1.73	2.53	2.74	1.57
技能创建（相对初始参数化知识技能的差值）											
+Error	1.53	1.10	0.55	7.07	6.98	3.62	1.63	1.22	13.30	18.93	3.24
+Success	1.89	0.91	1.45	5.25	9.60	4.86	2.39	1.76	14.66	19.38	3.00
+Combined	3.04	0.90	1.10	6.07	9.69	3.04	1.48	1.13	13.52	19.72	2.99
技能编写模型：*Qwen3.5-35B-A3B*											
技能深化（初始技能：人工编写）											
+Error	11.50	1.31	2.31	1.40	0.63	5.11	3.00	1.12	0.67	1.46	1.10
+Success	15.81	4.63	2.42	1.69	0.84	0.76	2.47	1.16	1.18	2.90	1.32
+Combined	9.00	2.14	1.33	1.94	0.85	0.50	3.62	2.54	3.27	2.71	0.88
技能创建（初始技能：参数化知识）											
+Error	2.18	1.04	0.58	6.96	8.84	1.76	1.77	1.08	14.60	17.83	3.36
+Success	1.61	1.00	0.90	6.94	8.64	4.51	1.10	1.19	14.24	19.02	3.21
+Combined	2.75	1.02	1.28	6.37	8.19	5.51	1.62	1.91	10.98	19.64	3.16

Huichi Zhou, Siyuan Guo, Anjie Liu, Zhongwei Yu, Ziqin Gong, Bowen Zhao, Zhixun Chen, Menglong Zhang, Yihang Chen, Jinsong Li, Runyu Yang, Qiangbin Liu, Xinlei Yu, Jianmin Zhou, Na Wang, Chunyang Sun, and Jun Wang. Memento-skills: Let agents design agents, 2026a. URL https://arxiv.org/abs/2603.18743.

Yingli Zhou, Shu Wang, Yaodong Su, Wenchuan Du, Yixiang Fang, and Xuemin Lin. A comprehensive survey on agent skills: Taxonomy, techniques, and applications, 2026b. URL https://arxiv.org/abs/2605.07358.

A 实验细节

随机种子与算力。 除非另有说明，所有报告的结果均取三个随机种子 41、42 和 43 的平均值。实验运行在配备 8 块 NVIDIA A100 GPU 的节点上。全部实验与探索共消耗约 20,000 GPU 小时。

电子表格数据与评分。 SpreadsheetBench Verified 被划分为 200 个演化问题和 200 个留出测试问题；技能演化期间不使用任何留出样本。我们还在完整 SpreadsheetBench 上报告 Soft（子问题通过率）和 Hard（所有子问题均须通过）；其中移除了演化集中的全部问题，以确保技能演化期间见过的问题不会在测试时计分。对于 OOD 迁移，我们将 WikiTableQuestions 转换为基于组合式 Wikipedia 表格的电子表格任务，并将 HiTab 转换为同时需要层次化索引、隐式计算和语义推理的电子表格任务。评估集包含 200 个 SpreadsheetBench-Verified 留出样例、用于 Soft/Hard 的 2,529 个完整 SpreadsheetBench 样例（已移除演化集中的全部问题）、2,810 个 WikiTableQuestions 样例和 1,585 个 HiTab 样例。

表 8: 数学推理上的并行整合与顺序编辑比较（轨迹预算相同）。D-Test/AIME 的定义与 表 2 相同（使用模型为 122B 和 35B）。

条件	运行模型：*122B*		运行模型：*35B*		
	D-Test↑	**AIME↑**	**D-Test↑**	**AIME↑**	**时间↓**
无技能	92.0	90.4	89.0	83.3	0
顺序-B=4	93.0	**91.7**	89.0	68.3	3.8 分钟
顺序-B=1	94.0	85.8	90.0	81.7	25.9 分钟
并行	**95.0**	**91.7**	**94.0**	**88.3**	**2.0 分钟**

表 9: 在数学推理任务上，Trace2Skill 优于 ReasoningBank 检索。D-Test/AIME 的定义与 表 2 相同。

条件	运行模型：*122B*		运行模型：*35B*	
	D-Test↑	**AIME↑**	**D-Test↑**	**AIME↑**
无技能	92.0	90.4	89.0	83.3
ReasoningBank（仅错误）	94.0	90.8	91.0	80.8
ReasoningBank（组合）	94.0	90.4	91.0	80.4
Trace2Skill +Error（本文）	**95.0**	**91.7**	**94.0**	**88.3**

表 10: 在 DocVQA 上使用 +Combined 轨迹池时，并行整合与顺序编辑的比较。ANLS/Acc 的定义与 表 3 相同。

设置	运行模型：*122B*		
	ANLS↑	**Acc↑**	**时间↓**
无技能	0.6300	70.27	0
顺序-B=4	0.8674	90.80	5.2 分钟
顺序-B=1	0.8694	91.05	35.9 分钟
并行	**0.8833**	**92.52**	**4.8 分钟**

表 11: 在 DocVQA 上，Trace2Skill 优于 ReasoningBank 检索。ANLS/Acc 的定义与 表 3 相同。

设置	运行模型：*122B*		运行模型：*35B*	
	ANLS↑	**Acc↑**	**ANLS↑**	**Acc↑**
无技能	0.6300	70.27	0.6582	73.17
ReasoningBank（组合）	0.8668	90.90	0.8568	89.62
Trace2Skill +Combined（本文）	**0.8833**	**92.52**	**0.8740**	**92.00**

基线技能。 Human-Written 是 Anthropic 官方的 `xlsx` 技能；Parametric 则是仅基于参数化知识提示 Qwen3.5-122B-A10B、在没有轨迹依据的情况下生成的 `xlsx-basic` 种子。

模型服务与演化协议。 两个主要的技能作者/用户模型是 Qwen3.5-122B-A10B 和 Qwen3.5-35B-A3B。我们使用 Hugging Face 检查点 Qwen/Qwen3.5-122B-A10B 和 Qwen/Qwen3.5-35B-A3B。二者都是 instruct/think 混合式 MoE 模型：多轮 ReAct 风格智能体使用 instruct 模式，层次化合并、成功分析和补丁转换等单次调用步骤使用 thinking 模式。模型通过 vLLM [Kwon et al., 2023] 提供服务；对于推理模式调用，我们遵循 Qwen 官方推荐的解码设置。阶段 1 为每个问题生成一条轨迹；阶段 2 并行运行 128 个子智能体，合并批大小为 32。我们不另设工具调用预算。

B 补充分析结果

B.1 延迟分析

当工作进程数 W=128、错误经验数 $N\approx$70 时，所有分析智能体可在一轮中并行执行。在合并批大小 B_{merge}=32 时，层次化合并仅额外增加 $\lceil \log_{B_{\text{merge}}} N \rceil \approx 2$ 轮顺序调用，每个合并层一轮，因此总计约为 $\approx$3 轮顺序 LLM 调用。两个顺序基线分别需要 N 轮和 $\lceil N/B \rceil$ 轮，因为每次技能编辑都依赖前一次编辑。在实践中，并行整合耗时 3 min，Seq-B=1 耗时 60 min（慢 20×），Seq-B=4 耗时 15 min（慢 5×），且差距随 N 线性扩大。所有时间均为在同一种 8-GPU A100 节点配置上测得的技能生成实际耗时。

B.2 数学推理

表 8、9 将正文中的分析消融扩展到数学推理。这些数学消融使用 +Error 设置，因为它是 表 2 中最强的数学设置。并行整合在所有报告的数学指标上取得最佳或并列最佳结果，同时技能生成的实际耗时也最低。

B.3 DocVQA

表 10、11 将正文中的 DocVQA 结果扩展到顺序范式与检索记忆的对比。这些 DocVQA 消融使用 +Combined 设置，因为它是 表 3 中最强的 DocVQA 设置。 表 10 单独考察了 122B 组合协议的批处理消融：所有非基线行都使用相同的 25 条失败轨迹和 25 条成功轨迹，区别仅在于顺序批大小与并行整合。这 50 个 DocVQA 样例仅用于演化，不纳入评估；所有 DocVQA 分数均在其余 5,299 个留出样例上计算。正如 附录 B.1 所分析的，并行与顺序范式之间的效率差距会随轨迹数量增加而扩大。

B.4 跨模型轨迹归纳

图 6 展示了这一跨模型轨迹归纳设置。使用 Qwen 轨迹时，由 Gemma-4-31B-it 和 GPT-5.5-high 编写的 Deepening 技能均使 Qwen3.5-122B 用户模型优于 No Skill（27.67% Vrf）和 Human-Written（48.33% Vrf）。最强设置是由 GPT-5.5-high 编写的 Deepening +Combined，达到 68.00% Vrf，表明跨模型轨迹证据仍可归纳出有意义且可迁移的技能。

实现细节。 我们使用 vLLM [Kwon et al., 2023] 为 Gemma-4-31B-it 提供服务，并采用其 Hugging Face 模型卡中的官方推荐生成配置（google/gemma-4-31B-it）；在 ReAct 智能体运行中禁用推理。GPT-5.5-high 通过 OpenAI 官方 API 访问，仅将推理强度设为 high，其余设置均保持默认值。

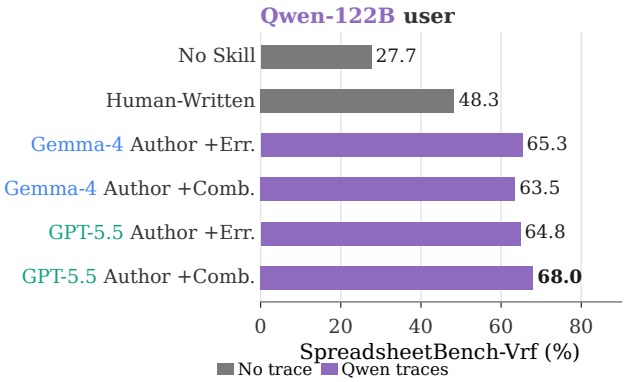


图 6: SpreadsheetBench-Vrf 上的跨模型轨迹归纳（通过率，%）。Gemma-4 和 GPT-5.5-high 基于 Qwen 轨迹编写技能，随后由 Qwen3.5-122B 用户模型运行。

C 定性分析

C.1 基于智能体的错误分析与 LLM 错误分析对比

我们对 33 个同时由基于智能体的错误分析器 $\mathcal{A}^-$ 和单次调用 +Error LLM 基线分析的共有错误案例进行了定性审查。两个流程仅在 4 个案例（12.1%）上高度一致，而在 18 个案例（54.5%）上对根因或恰当的技能补丁存在明显分歧。主要差异在于能否访问证据：$\mathcal{A}^-$ 可以检查输入/输出制品、将提交答案与真实答案对比并验证候选修复，而 +Error LLM 只能依据执行日志推断失败原因。

这一限制使纯 LLM 分析器容易产生日志层面的假阳性。在出现解析错误消息的案例中，+Error LLM 有 57% 将解析错误归为主要根因，而基于智能体的分析器的这一比例为 14%。在一条具有代表性的轨迹中，尽管制品评估表明输出已经正确，+Error LLM 仍臆造了三个不同的失败原因。相比之下，基于智能体的循环可以在检查生成文件并重新运行相关验证步骤后排除此类解释。

这些定性差异解释了为何基于智能体分析得到的补丁在 第 4.1 节 中迁移得更可靠。$\mathcal{A}^-$ 不会编码日志中的表面症状，而是将补丁锚定到经验证的失败机制：目标范围错误、公式值过期、工作簿结构损坏、类型转换或缺少回读验证。由此产生的编辑更可能成为领域通用的防护规则，并且在作为可迁移技能应用时，更不容易降低 ID、跨模型或 OOD 设置下的表现。

C.2 补丁选择定性分析

与补丁无关的退化。 后续迭代补丁确实会改变智能体在某些任务上的行为，并按补丁意图将结果从错误翻转为正确。然而，同样的编辑也会改变与目标失败无关的行为，使其他原本正确的任务从正确翻转为错误。这些翻转大多与来源任务无关：在十次迭代的贪心 +Combined 路径（种子 41）中，所选补丁的来源任务与评估划分完全不重叠，因此某一步破坏的原本正确任务从未被所选补丁针对；这属于副作用，而非针对性退化。因此，每个实际生成的步骤都记录了两个方向的大量翻转（例如，某一步有 57 个失败转通过和 21 个通过转失败，另一步则分别为 26 和 46），所以每次添加都会同时修复和破坏任务。我们仔细检查了这些翻转，发现大多数失败转通过与补丁相关，而通过转失败通常与补丁无关。由于每个贪心步骤都会同时修复和破坏任务，净准确率不再上升，而是进入平台期。因此，局部最优的单个补丁无法组合成全局增益：补丁价值取决于各次添加之间的交互，因而必须优化补丁的组合效应，而非其彼此孤立的边际贡献。

语义重叠。 相同的失败模式会跨迭代反复出现，因此流程会重复提出语义重叠、针对相同电子表格行为的补丁，包括公式重算、工作簿结构保留、引用整合和验证清单。贪心路径具体体现了这一点：在十次迭代中，选择器不断回到少数几个主题，包括重算与验证（第 1、6 次迭代）、结构/表头/工作表处理（第 2、9 次迭代），以及格式/类型/日期处理（第 4、5、7、8 次迭代）。于是，从已覆盖主题中新选出的补丁主要只是在重述技能中已有的指导，带来的边际增益很小。

C.3 学得的 SoP 细节

我们检查了 122B Deepening +Combined 运行产生的 323 个映射补丁。四种最常见的 SoP 分别被 323 个补丁中的 55.1%、54.8%、42.7% 和 16.4% 引用[2]；由于单个补丁可以引用多个主题，这些比例之和超过 100%。

公式重算与写回验证（占补丁的 55.1%）。 每次写入公式后运行 `recalc.py`，并使用 `data_only=True` 重新打开文件以确认计算结果；跳过此步骤会使单元格保留过期值，这是该次运行中最常见的单一错误模式。

[2]Trace2Skill 的合并过程会追踪用于归纳合并补丁（SoP）的原始补丁。

工具选择：优先使用 `openpyxl` 而非 `pandas.to_excel()`（占补丁的 54.8%）。 使用 `pandas` 执行读取/转换逻辑，使用 `openpyxl` 写回；先将输入文件复制到输出路径，以保留所有结构锚点。`pandas.to_excel()` 会悄然破坏公式关系和命名范围。

显式回读验证（占补丁的 42.7%）。 写入后重新打开输出文件，并在提交前确认每个目标单元格都保存了预期值；典型的失败轨迹会遗漏这项检查。

结构编辑安全性（占补丁的 16.4%）。 按降序删除行，以防索引偏移导致损坏；编辑前复制输入工作簿，以保留格式和公式。错误轨迹记录了这两种失败模式，成功轨迹则证实了这一防护工作流。

小众特殊情况被路由到 `references/`。 支持度较低的观察不会被丢弃，而是被路由到 13 个补充参考文件中，而非主 `SKILL.md`。例如，单元格颜色提取以及特殊业务逻辑下 FIFO 与 LIFO 不匹配的问题会被放入按需引用的参考文件。这与成熟的技能设计实践一致：过程性指导从通用规则逐步深入到具体案例，主文档编码通用工作流规则，而 `references/` 充当低频边界情况的按需查询层。Trace2Skill 从轨迹证据中自动恢复这一层次结构，无需人工整理。

其他中等支持度的 SoP。 以下 SoP 在同一次运行中获得中等程度的支持（约占补丁的 3.1%–4.6%），也被编码进演化后的技能。

目标范围与答案位置验证（占补丁的 4.6%）。 写入前，根据任务元数据核实准确的目标工作表名称、单元格范围和 `answer_position` 字段。误读这些字段——写入错误的工作表或偏移一位的范围——会导致静默失败：没有错误消息，却得到零分。

数据类型与日期时间保留（占补丁的 4.6%）。 将日期和数值写为原生 Python 类型，而非字符串。`pandas` 日期解析和 `openpyxl` 单元格赋值都可能悄然将日期时间值字符串化；写入前应检查每一列的 dtype，并使用 `openpyxl` 的原生日期时间赋值。

编辑前探索工作簿结构（以成功为主，约占补丁的 ∼4.0%）。 在执行任何写入前列出所有工作表、检查行列布局并核实表头位置。这种编辑前探索可避免工作表错误和范围错误，在该次运行中 151 个偏向成功的补丁里占据相当大的比例。

D 基于智能体的错误分析完整结果

表 12 将正文中的 Avg 对比扩展为逐数据集结果，覆盖两种技能作者模型、两种演化模式和两种技能用户模型。主要规律是，基于智能体的错误分析更为稳定：+Error（我们的方法）在四个作者模型–模式分块中的三个取得更高 Avg，并在大多数 SpreadsheetBench 列上获胜。例外是由 122B 编写的 Creation，其中 +Error LLM 的 Avg 略高，因为它在 OOD 表格基准上表现更好；即便如此，基于智能体的分析器在分布内 SpreadsheetBench 指标上仍然更强。总体而言，完整表格支持正文的核心论点：相比仅分析日志，检查制品并验证修复所产生的错误补丁迁移得更可靠。

E 选择性补丁聚合细节

主要实验在层次化整合后应用所有学得的补丁；选择性聚合则考察某个补丁子集能否产生更好的技能。我们沿用 第 2.1 节 的形式化定义：$\mathcal{S}$ 表示技能，π_θ 表示固定智能体，$\mathcal{P}$ 表示阶段 2（第 2.3 节）产生的轨迹级补丁池，$c(p)$ 表示支持补丁 $p \in \mathcal{P}$ 的来源轨迹数量。我们用 $\mathcal{S} \oplus \mathcal{P}'$ 表示通过阶段 3 的 diff 应用（第 2.4 节），将补丁子集 $\mathcal{P}' \subseteq \mathcal{P}$ 应用于 $\mathcal{S}$ 后得到的技能；用 $\mathrm{acc}_V(\mathcal{S}) \triangleq \mathcal{P}(\mathcal{S}; \pi_\theta, V)$ 表示其在含 32 个任务的验证集 V 上的验证通过率。下面两个选择器都搜索一个子集，使实际生成的技能最大化 acc_V。

贪心 top-1 选择。 在第 t 次迭代中，我们评估当前技能 $\mathcal{S}_t$，收集新提出的补丁 $\mathcal{P}_t$，并从满足 $c(p) \geq 5$ 的补丁中选取覆盖度最高的五个，组成候选集 $\mathcal{C}_t$。对于每个 $p \in \mathcal{C}_t$，我们通过添加一个和移除一个的干预来估计其贡献：

$$\begin{aligned}\Delta^+(p) &= \mathrm{acc}_V(\mathcal{S}_t \oplus p) - \mathrm{acc}_V(\mathcal{S}_t),\\ \Delta^-(p) &= \mathrm{acc}_V(\mathcal{S}_t \oplus \mathcal{P}_t)\\ &\quad - \mathrm{acc}_V\big(\mathcal{S}_t \oplus (\mathcal{P}_t \setminus \{p\})\big).\end{aligned}$$

丢弃满足 $\Delta^+(p) < 0$ 或 $\Delta^-(p) < 0$ 的补丁；其余补丁按 $r(p) = \Delta^+(p) + \Delta^-(p)$ 排序，覆盖度用于打破平局。对于排名最高的补丁 $p_t^\star$，下一技能为 $\mathcal{S}_{t+1} = \mathcal{S}_t \oplus p_t^\star$。 第 4.3 节 报告了 SpreadsheetBench-Vrf 结果。

补丁子集上的贝叶斯优化。 BO 从阶段 2 的补丁池（第 2.3 节）中固定一个补丁全集 $\mathcal{P} = \{p_j\}_{j=1}^m$，保留覆盖度最高的 $m \leq 15$ 个补丁。它从初始技能 $\mathcal{S}_0$ 出发，搜索二元包含向量 $x \in \{0,1\}^m$，并用 $\mathcal{P}_x = \{p_j : x_j = 1\}$ 表示所选子集。目标函数为

$$f(x) = \mathrm{acc}_V(\mathcal{S}_0 \oplus \mathcal{P}_x) - \lambda \|x\|_0, \qquad \lambda = 0.$$

初始设计评估空子集、所有单元素子集、五个随机混合子集以及完整子集。每个批次结束后，已评估子集按 $f(x)$ 被划分为优集 G 和劣集 B：前者包含排名前 $\gamma = 0.2$ 的部分，后者包含其余子集。采用 Beta 平滑 $\alpha = 1$

表 12: 基于智能体的错误分析（+Error）与单次调用的 +Error LLM 在技能深化、技能创建以及所有 122B/35B 编写模型–使用模型组合上的比较。Vrf/Soft/Hard、OOD 划分（WikiTQ/HiTab）和 Avg 的定义与 表 1 相同。每列两种方法中较优者以**粗体**标出。

	技能使用模型：*Qwen3.5-122B-A10B*					技能使用模型：*Qwen3.5-35B-A3B*					
	SpreadsheetBench			*OOD*		*SpreadsheetBench*			*OOD*		
条件	**Vrf**	**Soft**	**Hard**	**WikiTQ**	**HiTab**	**Vrf**	**Soft**	**Hard**	**WikiTQ**	**HiTab**	**Avg**
技能编写模型：*Qwen3.5-122B-A10B*											
技能深化											
+Error（本文）	65.83	**46.60**	**27.43**	**76.30**	**43.45**	**36.67**	**22.47**	**6.23**	**18.28**	**14.85**	**36.21**
+Error LLM	**67.00**	43.93	25.23	39.81	36.68	25.00	22.43	6.23	11.24	9.74	28.00
技能创建											
+Error（本文）	**49.00**	**40.37**	**23.37**	31.62	15.66	**28.83**	**23.23**	**7.87**	22.20	23.58	26.02
+Error LLM	27.17	27.73	16.20	**47.26**	**30.26**	19.83	17.60	4.70	**23.30**	**36.53**	**26.60**
技能编写模型：*Qwen3.5-35B-A3B*											
技能深化											
+Error（本文）	**65.00**	**44.80**	**25.17**	68.32	38.93	27.00	**22.20**	8.20	**11.73**	**13.38**	**32.58**
+Error LLM	37.83	22.93	12.83	**77.05**	**42.05**	**30.50**	20.17	**8.73**	9.95	12.73	28.80
技能创建											
+Error（本文）	**27.17**	**28.90**	**18.53**	**81.38**	**45.61**	**24.00**	**21.00**	**6.53**	**32.80**	31.75	**34.45**
+Error LLM	22.00	27.67	16.60	54.61	37.93	23.50	16.87	4.93	11.24	**33.75**	26.49

时，每个补丁的 Bernoulli 包含率为

$$\mu_j^G = \frac{\sum_{x\in G} x_j + \alpha}{|G| + 2\alpha}, \qquad \mu_j^B = \frac{\sum_{x\in B} x_j + \alpha}{|B| + 2\alpha}.$$

候选子集按 $\rho = 0.1$ 从 $\phi_j = (1-\rho)\mu_j^G + \rho/2$ 中采样，并依据 TPE 采集函数排序：

$$A(x) = \sum_{j=1}^{m} \left[\log q(x_j; \mu_j^G) - \log q(x_j; \mu_j^B)\right], \qquad q(b;\mu) = \mu^b(1-\mu)^{1-b}.$$

每轮从包含 500 个候选项的池中评估排名前八的候选项，在完成四轮或连续三轮没有改进后停止，并返回观察到的最佳子集。 第 4.3 节 将 BO 选择的 +Error 技能与应用全部 +Error 补丁进行比较。

F 更广泛应用的细节

我们将 Trace2Skill 扩展到三种更广泛的文档智能体设置：PDF 提取、PPTX 编辑和 DOCX 编辑。所有设置均以 Anthropic 官方文档技能作为底层技能族 [Anthropic, 2026a]。源轨迹仅用于技能演化，留出目标任务仅用于评估。每项评估都使用任务对应的精确本地验证器，并报告与主要实验相同的任务级通过率指标。 表 13 总结了正文中各模态使用的迁移设置；“Base”表示应用所列源轨迹之前的直接技能。

PDF 提取。 PDF 研究将 VRDU 和 VAREX 改编为本地 PDF-to-JSON 任务 [Wang et al., 2023b, Barzelay et al., 2026]。我们使用 VRDU Registration Forms 作为源语料库，因为它提供了一个连贯的真实集合，其中包含视觉内容丰富的表单提取轨迹，并反复出现相同的字段和布局约定。我们在 VAREX Flat 上进行评估，将其作为独立的留出结构化提取基准，以检验归纳出的表单提取流程能否跨数据集迁移，而非仅迁移到同一语料库中的更多样例。报告的 PDF 分数使用经验证后最强的 VRDU Registration 源划分；其他 Registration 划分也带来正向效果，但增益较小。

PPTX 编辑。 PPTX 研究使用 Talk-to-Your-Slides 中的 TSBench 演示文稿编辑任务 [Jung et al., 2026]。 PPTX 结果基于一组演示文稿上的 TSBench 训练轨迹进行演化，并在由留出演示文稿构成、与训练演示文稿互不重叠的 TSBench OOD 划分上评估，从而检验跨演示文稿文件的迁移，而非对同一组演示文稿执行更多编辑。

DOCX 编辑。 DOCX 研究使用 OfficeBench 作为留出目标，因为它包含具有可验证 Word/DOCX 输出的真实办公自动化任务 [Wang et al., 2024a]。我们将全部 64 个可转换的 OfficeBench DOCX 子任务仅保留用于在 `max_turns=100` 设置下评估。预留该目标集后，剩余真实且可验证的 DOCX 智能体数据过少，因此我们使用生成的办公文档轨迹作为源分布。这些轨迹覆盖可复用操作，例如编辑模板、保留文档结构、跨文件追加内容以及生成所需的附属文件。

领域	源轨迹	留出评估	Base	Trace2Skill	增益
PDF	VRDU Registration Forms	VAREX Flat 验证集	76.9%	85.3%	+8.4 pp
PPTX	TSBench 训练轨迹	TSBench 演示文稿互斥 OOD	72.5%	88.8%	+16.3 pp
DOCX	生成的文档操作任务	OfficeBench DOCX 留出集	79.7%	87.5%	+7.8 pp

表 13: 更广泛的文档智能体迁移。每一行都使用源轨迹演化一种 Anthropic 官方文档技能，并在留出目标领域上评估。Base 和 Trace2Skill 为目标通过率（%）；增益是二者相差的百分点（pp）。

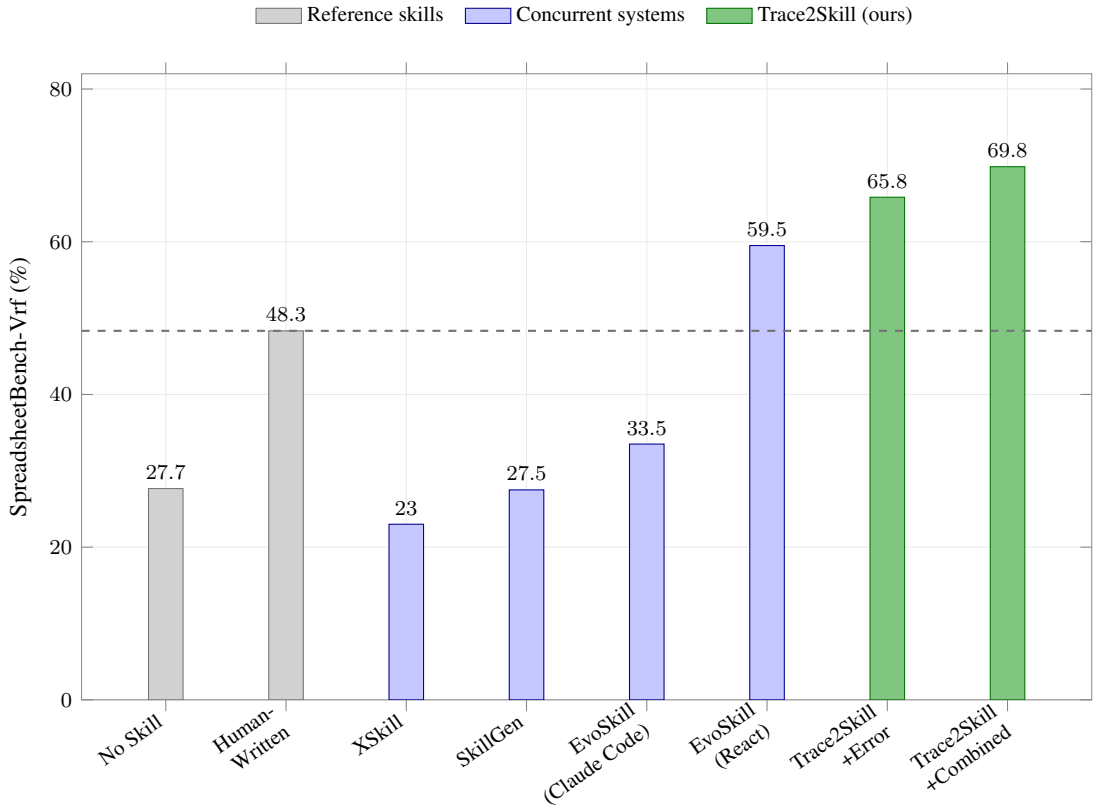


图 7: SpreadsheetBench-Verified 上的正面对比（Vrf，通过率 %）：所有系统均使用同一开放基础模型 Qwen3.5-122B-A10B 和留出的 200:400 测试切片。柱形涵盖 No Skill 下限、Human-Written 初始技能（虚线）、三个复现的同期系统和 Trace2Skill Deepening。EvoSkill 的两根柱均来自同一套忠实复现，唯一差异是智能体运行框架（遵循原论文的 Claude Code，与匹配我们设置的 React）。

G 与同期技能演化系统的正面对比

本附录是对 第 4.1 节 中同等条件对比的正面补充：它将 Trace2Skill 与三个完整的同期技能演化系统——XSkill [Jiang et al., 2026]、EvoSkill [Alzubi et al., 2026] 和 SkillGen [Ma et al., 2026]——直接比较，而不是每次只分离一个设计选择。我们出于三个原因没有将其放入正文。**归因**：完整系统的分数会混合设计思想、基础模型、运行框架和工程实现，因而无法判断哪个因素造成了差异；相比之下，第 4.1 节 在共享的轨迹池、模型和运行框架下仅改变一个设计选择，这才为我们的因果结论提供依据。**适用范围**：这些系统追求不同目标，并围绕特定的（通常为专有的）基础模型、定制运行框架和不同任务领域构建；而我们有意研究能够自我演化的开放模型。将这些系统移植到同一个基准上，只能展示它们在我们的设置中的行为，并不代表其设计上限，因此我们对这些数字作保守解读。

我们按照各系统官方代码库复现这些流程，并将其适配到同一套共享协议：使用由 vLLM 提供服务的同一开放基础模型 Qwen3.5-122B-A10B，在 SpreadsheetBench-Verified 的 0:200 切片上演化，并在留出的 200:400 切片上评估，采用官方实例准确率（即我们的 Vrf 指标）计分。 图 7 报告了对比结果，参照项包括 No Skill 下限、若干系统同样采用的 Human-Written 初始技能，以及 Trace2Skill Deepening +Error/+Combined。 Trace2Skill 取得最高 Vrf（+Error 为 65.83，+Combined 为 69.83），超过其他三个系统；各系统的具体设置及其与原生流程的偏差详述如下。

XSkill. XSkill 是一种非参数记忆方法，它累积任务级过程技能和动作级经验，随后在推理时进行检索、适配

用户/轨迹来源	Deepening			Creation		
	基础	skill-creator	Trace2Skill +Error	基础	skill-creator	Trace2Skill +Error
122B/122B	48.33	27.33	65.83	26.17	26.67	49.00
122B/35B	48.33	19.50	65.00	26.17	18.33	27.17
35B/35B	9.67	18.50	27.00	20.17	17.67	24.00
35B/122B	9.67	28.00	36.67	20.17	27.33	28.83
平均值	**29.00**	**23.33**	**48.63**	**23.17**	**22.50**	**32.25**

表 14: 与 Anthropic `skill-creator` 基线在 SpreadsheetBench-Verified（Vrf）上的对比（通过率，%）。对于每个技能用户/轨迹来源配对，分别在 Deepening 和 Creation 下将 `skill-creator` 与对应的基础技能及 Trace2Skill +Error 对比；最后一行给出各列平均值。

并注入；该方法面向使用工具的多模态智能体而设计。我们在 0:200 切片（与 Trace2Skill 相同的演化集）上收集 200 条轨迹，通过分批 LLM 合并将每条轨迹对应的技能合并为一份包含 865 个英文单词的全局 `SKILL.md`，并蒸馏出 20 条经过筛选的动作级经验。测试时，对于每个实例，我们按词汇重叠检索 top-3 经验，让 LLM 根据任务适配全局技能，将其注入系统提示词，并让智能体最多运行 100 轮。该复现达到 23.0 Vrf。我们还用 Qwen3-Embedding-0.6B 替换了词汇检索器，但性能略差，仅为 20.0。

EvoSkill. EvoSkill 是一个自我演化循环，由基础智能体、失败提议器、技能生成器、验证评估器和有界前沿组成，面向搭配 Opus 4.5 的 Claude Code 运行框架而设计。我们在 0:160 上挖掘失败，在 160:200 上验证候选项，并在 200:400（我们的 SpreadsheetBench Vrf 测试子集）上评估所选程序，前沿大小为 3；每次迭代都会提出并生成一份替换用的 `xlsx/SKILL.md`，再在验证切片上评分，Qwen3.5-122B-A10B 由 vLLM 提供服务。我们运行了两种框架：与我们设置匹配的 React 风格本地框架达到 59.5 Vrf（5 次迭代）；通过 Anthropic 兼容代理访问的 Claude Code（最接近原生框架）达到 33.5（8 个工作进程，在接近第 11 次迭代时提前停止）。

SkillGen. SkillGen 在留出评估前通过成对验证门，根据对比式失败/成功分析归纳出紧凑技能。我们在 0:200 上收集基线轨迹，使用 Qwen3-Embedding-0.6B（k-means）聚类失败与成功摘要，以综合得到对比式观察，再生成一份紧凑的 `SKILL.md`；随后通过净增益接受门在 0:200 上进行最多 8 轮优化和验证（我们使用 `min_net_gain_abs`$= 1$；上游默认值为 ≥ 3），并在 200:400 上评估被接受的技能。训练切片上的基线为 52.5，第 1 轮净增益为 0（拒绝），第 2 轮为 -4（拒绝），第 3 轮为 $+1$（接受），最终在留出切片上得到 27.5 Vrf。训练集有所改进而测试集发生退化，这可能源于分布偏移：训练切片混合了 125 个工作表级任务和 75 个单元格级任务，而测试切片仅包含单元格级任务。

这些正面对比补充而非取代 第 4.1 节 中不含混杂因素的同等条件对比；我们的主张以后者为依据。

H Skill-Creator 基线

对于仅评估 Vrf 的外部基线，我们通过搭载 Opus 4.6 medium 的 Claude Code，使用 Anthropic 官方 `skill-creator` 技能进行技能起草与改进 [Anthropic, 2026c]。表 14 报告了 SpreadsheetBench-Verified Vrf 分数，采用与主要电子表格评估相同的技能用户/轨迹来源配对，并列出对应的 表 1 基线和 Trace2Skill +Error 结果。无论是 Deepening 还是 Creation，所得技能的平均表现都未能改善对应的基础技能，而 Trace2Skill +Error 始终更强。

该基线使用的提示词如下：

Skill Creator 基线提示词：Deepening

```
我有一个使用 xlsx 技能执行电子表格任务的智能体。
智能体轨迹位于 /path/to/traces/，其中错误轨迹和成功轨迹
分别标注为 *_FAILURE.md 和 *_SUCCESS.md。
你的任务是深化 xlsx 技能（/path/to/xlsx），以提高智能体
未来使用新技能时的表现。你应首先从轨迹中归纳共同且可
泛化的失败与成功模式，然后使用 skill-creator 技能修补
xlsx 技能。
```

Skill Creator 基线提示词：Creation

```
我有一个执行电子表格任务的智能体。
智能体轨迹位于 /path/to/traces/，其中错误轨迹和成功轨迹
分别标注为 *_FAILURE.md 和 *_SUCCESS.md。
你的任务是创建一个电子表格技能，以提高智能体配备该技能
后的未来表现。你应首先从轨迹中归纳共同且可泛化的失败与
成功模式，然后使用 skill-creator 技能创建该技能。
```

I 提示词模板与中间输出

本附录复现流程各阶段使用的关键提示词模板，并展示具有代表性的中间输出，使整个流程完全透明且可复现。

I.1 阶段 1：智能体系统提示词模板

在收集轨迹期间，智能体 π_θ 使用以下系统提示词运行。推理时，技能 $\mathcal{S}_0$ 会被置于用户上下文之前。请注意，这与标准技能加载流程不同：后者起初只允许智能体访问技能描述。我们将 SKILL.md 内容预加载到系统提示词中，以此简化流程，因为 Trace2Skill 专注于改进一个相对于任务已知的固定目标技能。因此，不需要标准的技能选择步骤。需要强调的是，使用 Trace2Skill 技能的智能体仍需按流程发现预加载 SKILL.md 所引用的资源（例如资源文件和脚本），这些资源本身不会被预加载。

阶段 1——智能体系统提示词（节选）

角色：你是一名专家级 role（例如电子表格分析）智能体。
技能上下文：［在此插入 $\mathcal{S}_0$ 的内容］
任务：描述任务和输入文件。
可用工具：描述工具和环境。例如，在可访问文件系统的 ReAct 中使用 `bash`（执行 shell 命令）。
格式：采用 ReAct 风格交互——在推理轨迹与工具调用之间交替进行，直至完成任务。

I.2 阶段 2：分析器提示词模板与补丁示例

在阶段 2 中，提出补丁的智能体首先提取类似于 [Ouyang et al., 2026] 的错误与成功记忆条目，即可能有助于未来执行任务、且可泛化的轨迹级知识。随后，智能体读取原始技能目录，再提出补丁，将这些记忆条目编码进技能。

I.2.1 错误分析器提示词（$\mathcal{A}^-$）

错误分析器系统提示词（节选）

角色：你是一名面向 {domain} 任务的专家级失败分析智能体。

使命：给定智能体的执行制品（日志和生成文件）以及真实解答，诊断智能体为何失败，找出具有因果性的失败原因，并实施最小修复，使智能体输出与真实答案一致，以此 验证你的诊断。你的分析必须**系统化**、**由证据驱动**且**可复现**。能够验证时，**不要猜测**。

必要工作流（必须遵守）：
1. 理解任务和失败表象——准确指出输出中的问题。
2. 将失败追溯到智能体行为——定位造成不匹配的决策或代码步骤。
3. 用最小修复验证根因——写出修复后的输出，并对照真实答案重新评估。
4. 重新评估——若仍然失败，返回步骤 1–3 并修正诊断。

输出：生成：(1) 失败原因条目——以可观察的智能体行为为依据、系统化且具有因果性的原因；(2) 失败记忆条目（≤3）—— 智能体为避免类似失败而应记住的可泛化见解。

I.2.2 成功分析器提示词（$\mathcal{A}^+$）

成功分析器系统提示词（节选）

角色：你是 AI 智能体系统成功模式分析方面的专家。

使命：给定一条成功的智能体轨迹，识别促成正确答案的可泛化 行为模式。

要求：
广泛覆盖——轨迹中的每一种有效行为都必须由某个模式涵盖。
频率感知——覆盖更多实例的模式应优先列出；罕见行为应归入最接近的更广泛模式。
泛化性——每个模式都必须描述通用机制，而非某个任务特有的单一细节。

输出：一组紧凑的成功记忆条目，包含标题、描述和所观察到的有效行为的具体示例。

I.2.3 轨迹级补丁示例（错误分析器输出）

每个分析器针对一条轨迹生成一个结构化补丁。以下示例来自错误分析器（$\mathcal{A}^-$）对一条失败轨迹的处理，其中智能体删除了指定范围之外的行。该补丁建议在 `SKILL.md` 中新增一节，说明行操作的安全性；随后，这

个补丁与另外 52 个涉及相关结构编辑失败的补丁一起被整合为“结构编辑安全性”SoP（第 4.2 节）。

阶段 2 轨迹级补丁示例（错误分析器，单条轨迹）

分析器推理：单条错误记录揭示了清晰的失败模式：智能体删除了指定范围之外的行，且无法区分部分为空的数据行与完全为空的填充行。该技能缺少有关行/列操作或范围边界约束的任何指导。

```
-- a/SKILL.md
+++ b/SKILL.md
@@ -55,0 +55,26 @@ ## Formula Verification Checklist
+### 行与列操作
+
+#### 范围边界约束
+- 当任务指定单元格范围（例如 F14:M83）时，操作
+  必须严格保持在这些边界内
+- 解析范围格式：start_col + start_row : end_col + end_row
+- 切勿对范围之外的行/列执行操作
+
+#### 行删除逻辑
+删除任何行之前，须同时验证以下两个条件：
+1. 目标列为空（以任务要求为准）
+2. 该行位于明确的删除范围内
+
+除非任务明确要求，否则**不要删除**所有列均完全为空的行——
+它们通常用作结构性填充。
+
+#### 操作后验证
+完成行/列操作后：
+- [ ] 剩余行数与预期数量一致
+- [ ] 范围外的表头/表尾/填充行均未受影响
+- [ ] 相邻列中的数据完整性得以保留
+- [ ] 遵守范围边界（无偏移一位错误）
```

I.3 阶段 3：合并算子提示词与整合补丁示例

I.3.1 合并算子提示词（$\mathcal{M}$）

合并算子系统提示词（全文）

你是一名技能编辑协调者。你会收到多个独立提出的补丁，每个补丁都建议对技能文件夹作出修改。你的任务是将它们合并为一个连贯且无冗余的补丁。

准则：
1. **去重：**当多个补丁提出相同或非常相似的编辑时，保留最佳版本（最具体、措辞最佳）。
2. **解决冲突：**如果多个补丁对同一节提出相互矛盾的编辑，选择依据更充分的一项，或将二者综合为更好的编辑。
3. **保留独特见解：**不同补丁针对不同失败——纳入所有独特且不重复的编辑。
4. **保持简洁：**合并补丁的编辑量应不超过所有输入补丁中独特编辑量的总和。删除冗余内容。
5. **确保独立性：**合并补丁中的编辑必须在行级别相互独立——即使属于不同操作，也不得有两项编辑针对重叠行或同一段文本。
6. **原子化的创建/链接对：**针对 `references/*.md` 的 `create` 操作与在 `SKILL.md` 中插入指向该文件链接的编辑不可分割——要么都保留，要么都丢弃。

高频模式偏置：当多个补丁独立提出相似编辑，且都针对同一类失败或成功模式时，应将这种重复视为任务具有 系统性属性的证据。以更高优先级保留此类高频编辑，并将其表述为通用原则，而非特定实例的修复。

I.3.2 最终整合补丁 p^* 示例（完整合并层次之后）

以下节选展示了最终整合补丁 p^* 中的推理和代表性编辑；该补丁是在 SpreadsheetBench-Verified 上对 323 个独立轨迹补丁进行四层层次化合并后产生的。

阶段 3 最终补丁输出示例（节选）

```
{
"reasoning": "合并了 3 个同时涉及失败/成功证据的补丁。
关键整合决策：(1) 综合所有补丁中的 recalc.py 工作流，
采用最醒目的严重警告位置、CSV 回退验证和验证循环；
(2) 将库选择指导整合为完整的决策树；
(3) 合并行删除指导，强调所有补丁共同提出的
自底向上/从右到左删除顺序；
(4) 合并公式验证清单且不引入冗余。",
"edits": [
{
"file": "SKILL.md",
"op": "insert_after",
"target_section": "# Requirements for Outputs",
"content": "## 重要的自动化指导\ n\ n
```

```
**自动化应优先使用 Python 而非 VBA**：当任务要求
VBA 宏或电子表格自动化时，改用 Python 和
openpyxl/pandas 实现逻辑。这样能提供更好的错误处理、
更容易的调试、跨平台兼容性，并避免宏安全问题。"
},
{
"file": "SKILL.md",
"op": "insert_after",
"target_section": "## Important Requirements",
"content": "### 何时直接使用 Python 计算\ n
如果 LibreOffice/recalc.py 无法计算某些公式模式
（复杂数组公式、高级 INDEX/MATCH 组合），
则直接在 Python 中计算逻辑，并将结果写为值。
当公式语法正确却在重算后仍返回错误时，使用此回退方案。"
}
],
"changelog_entries": [
"添加了自动化指导，建议优先使用 Python 而非 VBA",
"将 recalc.py 警告整合为一条严重警告"
]
}
```

I.3.3 转换后的 diff 示例：最终整合补丁 p^*

阶段 3 生成的 JSON 补丁 p^* 会通过程序转换为 unified diff，并直接应用到技能目录。以下节选展示了 SpreadsheetBench-Verified 上最终应用的 diff 中的两个区块，分别对应最常见的两种 SoP：强制公式重算（占补丁的 55.1%）和工具选择指导（占补丁的 54.8%）。

阶段 3 应用 diff 示例（最终 p^* 节选）

```
-- a/SKILL.md
+++ b/SKILL.md
@@ -126,3 +261,18 @@ ## Common Workflow
 ## Common Workflow
+### 严重警告：必须执行公式重算
+
+**如果使用 openpyxl 向 Excel 文件写入任何公式，
+必须先运行 recalc.py，才能认为任务已经完成。**
+
+通过 openpyxl 写入的公式在重算前仅以文本字符串形式存在。
+如果不运行 recalc.py：
+- 使用 data_only=True 读取时，单元格返回 None/空值
+- 即使公式语法正确，评估也会失败
+- 输出文件不完整
+
+这项要求不可省略。在 recalc.py 确认成功前，
+不要继续验证或交付。
+
@@ -138,3 +285,9 @@ ### Working with openpyxl
+### 工具选择警告
+
+**严重警告**：修改含有需要保留的既有
+公式的电子表格时：
+- 使用 openpyxl（先 load_workbook()，再 save()）——公式会
+  保持为字符串
+- 避免使用 pandas（to_excel()）——它会悄然将公式永久转换为
+  静态值
```